

CS5228 Project Report

Twitter sentiment analysis: predicting the positive/negative sentiment of a tweet

[redacted] ([redacted])
[redacted] ([redacted])
[redacted] ([redacted])

November 12, 2017

1 Problem statement

In this project, the task of interest is to predict if a given Twitter tweet is expressing positive or negative sentiment. There are only two possible classes, therefore this is a binary classification task. Using a labelled dataset crawled from the micro-blogging service Twitter, we trained a model to predict positive/negative labels for tweets in a test dataset.

2 Highlights

We find that using Neural Network models yields the best results in terms of prediction accuracy, against other classifiers such as Support Vector Machines and Multinomial Naive Bayes. Using a voting-based model ensemble to combine multiple models provides a yet better result.

In terms of feature selection and engineering, we find that correcting misspelled words or abbreviated words within the features for training is crucial to improving the prediction accuracy of our model.

3 Dataset pre-processing

To create word-based features from tweets, we employed the Bag of Words model. Tweets were split into word tokens, which were used to form n-gram features, and for each tweet the occurrence of each feature is counted and weighted. The dataset can thus be represented as a matrix with one row per tweet and one column per feature (columns can be unigrams, bigrams, trigrams or other features).

Besides word-based n-grams, we also used four non-word-based features. We observed that these features appear to have a logarithmic distribution, which proved useful for feature-scaling. Hence, we appended the log-values of the following to the existing set of word-based features:

- Total number of dots in a tweet

- Number of single final fullstops, perhaps indicating a blunt tone. Here, fullstops must be followed by a space, or the end of tweet, and cannot be the end of some trailing dots.
- Total number of capital letters
- Total number of words

3.1 Data pre-processing

In particular, we performed the following preprocessing steps in the given order on the training set of 800,000 tweets:

- Extract all non-word features before further processing
- Convert all characters in tweet to lower case
- Split tweet into word tokens
- Remove characters within tokens that have been repeated consecutively more than 2 times
- For a tweet which contain the conjunctions, “but” and “however”, retain the phrase or sentence after the conjunctions, as this is deemed to be the main sentiment [4].
- Remove HTMLtags such as '&', URLs, usertags, numbers, but retain words in hashtags. Replace '<3' with '< 3' to represent a heart shape
- Extract emoticons into a list, using regular expressions
- Remove all punctuations which are not sentence-ending punctuations: {.,!?:;}
- Add Parts-of-Speech tags to the tokens
- Insert negation by finding negation words such as 'not' and negate all adverbs and adjectives behind it by appending the NOT_ tag, up to the next 5 words or to sentence-ending punctuation [2] [3].
- Pass the words and POS tags into a lemmatizer
- Remove the sentence-ending punctuations
- From the remaining tokens, parse the dataset again to form bigrams and trigrams
- Append the list of emoticons extracted previously back to tokens
- Add words into the feature vectors based on their sentence weights (words closer to the end has higher value, as the final sentiment tends to determine the overall sentiment)
- Correct short forms and leet spellings to original words (e.g. 2nite to tonight, any1 to anyone, cant to cannot)

3.2 Feature selection

After the pre-processing step, we remove features that occur only once, to reduce the dimensionality of the problem. We also normalized the data across rows (tweets) to account for larger values for longer tweets.

Next, we **compute the χ^2 values** for the features obtained thus far. Let n be the total number of tweets in the collection, $p_i(w)$ be the conditional probability of class i for documents which contain word, or feature, w , P_i be the global fraction of documents containing the class i , and $F(w)$ be the global fraction of tweets which contain the word w . The χ^2 -statistic of the word between word w and class i is then defined [1] as

$$\chi^2 = \frac{n \times F(w)^2 \times (p_i(w) - P_i)^2}{F(w) \times (1 - F(w)) \times P_i \times (1 - P_i)} \quad (1)$$

Plotting the $\log(\chi^2)$ graph, we observed an appropriate cutoff value of $\log(\chi^2)$ for which we can select a subset of the original features. We selected about 70,000 features.

4 Training and testing procedure

Cross-validation method To prevent overfitting, we used the k-fold cross-validation method, where we randomly split the training set into k sets, and iterated the validation set through the k sets, with the others as the training set. This was crucial to compare the evaluation metrics of different models in terms of generalization performance.

5 Experimental study and analysis

In terms of feature engineering, we experimented with **emphasizing words before exclamation marks** by repeating it to give it a bigger weight such as “My homework i need help!!”, or words that are in **capital letters**, e.g. “I LOOOOOOOOOVE thunderstorms!”, but this did not appear to be useful in improving accuracy.

We found that by **not removing stop-words** in the pre-processing step, we were able to capture more n-grams that can express sentiment in a tweet, which slightly improved our accuracy. The remaining stop-words that were less useful would have been removed by the χ^2 feature selection.

For feature selection, we explored using **Gain Ratio, Information Gain, and χ^2** to select features which have greater importance, and observe that selected features using χ^2 provided a model with slightly better accuracies.

For model parameter tuning, we employed a grid-search method. Also, by plotting the cross-validation and resubstitution errors against any single parameter, we could determine the best parameter values that balance between underfitting and overfitting the data.

6 Summary of project results and achievements

For the choice of classifiers, we explored the results of using a **Naive Bayes Classifier** (78.8%), **Support Vector Machine (with linear kernel)** (79.5%), **Logistic Regression** (78.0%), **Multi-Layer Perceptron Neural Network** (80.9%) and a **Recurrent Neural Network** (80.6%). We observed that using the Multi-Layer Perceptron results in higher accuracy than other classifiers.

We also experimented with **Model Ensemble**, where we combined the predictions of multiple models, in order to improve generalization performance and robustness. For the final submission (82.0%), we used a voting-based ensemble method, by computing the mode of predictions made by nine different best-performing classifiers.

We find that **correcting abbreviated words or mis-spelled words** to original words, e.g. “2nite” to “tonight”, “any1” to “anyone”, “cant” to “cannot”, “bday” to “birthday”, “sureprise” to “surprise”, resulted in a significant improvement in model prediction accuracy (e.g. SVM accuracy improved from 78.6% to 79.5%). In particular, the majority of the increase in performance comes from correcting the features that are relatively important, e.g. top 200 features, in terms of χ^2 values. However, it is of note that applying basic auto-correct tends to be ineffective, as short forms and leet spellings can have large edit distances away from the original words, and many valid “internet-style” words would be mistakenly corrected.

7 Possible Further Improvements

The following is a list of ideas we believe could help in improving model prediction accuracy:

Further token processing As text data, tweets are by nature noisy and informal, so further corrections may be done to improve the features extracted. Some possible avenues to explore in future would be:

- To correctly split concatenated words, and rejoin words that are broken by spaces and dashes
- To reduce tokens to phonetic representations, so that both creative and common misspellings may be mapped to the same words (e.g. “gr8” → “great”, “b-e-a-yooooo-tiful” → “beautiful”, “aloud” → “allowed”, “wannt” → “want”)
- To use keyboard layout information to facilitate autocorrection of common typo errors
- To employ regular expressions in the extraction of particular word forms, such as laughter and mashing of keyboard keys

Domains/authors While the original dataset does not contain details of authors or the domains to which the tweets belong, using such information should probably help in classifying the sentiment of tweets. Without these details, perhaps clustering tweets by specific domains (e.g. sports, music), or authors would allow the classifiers to perform better.

Handling neutral tweets Perhaps weights could be given to account for neutral tweets, or tweets which do not have sentiment as strong as others.

8 Future Directions

Handling complex sentiments Some of the most difficult tweets to classify express very complex sentiments, such as sarcasm, irony and dark humour. An example is the following: "...oh yeah and I've got a spare ticket... as usual!" This possibly expresses dark humour about being forever alone, but the word-based features by themselves may not capture this negativity. Even a human reader may not be confident of the expressed sentiment without further context. It would be interesting to explore ways to quantify and detect these complexities, perhaps through the use of sentence-level features, tracking of conversations and individual users' speech styles.

Deep learning Deep learning shows great potential for predicting sentiment in text. Further improvements can be made towards understanding semantics and sentence structure, by paying attention to the order and interaction of the words in a sentence. In particular, we find that recurrent neural network seems to have the greatest potential in a better prediction accuracy.

References

- [1] Medhat, W., Hassan, A., and Korashy, H. (2014) "Sentiment analysis algorithms and applications: A survey," *Ain Shams Engineering Journal*, vol. 5, no. 4, pp. 1093-1113.
- [2] Das, S. and Chen, M. (2001) Yahoo! for Amazon: Extracting market sentiment from stock message boards. Asia Pacific Finance Association Annual Conf. APFA
- [3] Farooq, U., Mansoor, H., Nongaillard, A., Ouzrout, Y., Qadir, M.A. (2017) "Negation Handling in Sentiment Analysis at Sentence Level", in *Journal of Computers*, Volume 12, Number 5, September 2017, pp. 470-478.
- [4] Agarwal, R., Prabhakar, T.V., Chakrabarty, S. (2008) "I Know What You Feel": Analyzing the Role of Conjunctions in Automatic Sentiment Analysis. *Advances in Natural Language Processing*, pp. 28-39.